

Performance determines whether different applications and services are communicating properly, waiting for data over the network, or whether clients are having trouble connecting or receiving information.

Performance issues affect the reliability of applications running in the system as well as the reliability of the entire environment. In most cases, a decline in performance is related either to latency or to bandwidth and hardware.

Since we cannot exceed the physical limit of the network environment, it is necessary to address performance issues with respect to specific service protocols and services like NFS, DNS, HTTP, etc. The limiting factors of the network and other related hardware provide the base line for the performance study. Once the base line is known, it is comparatively easy to identify the performance issues related to different service protocols.

After the correct diagnosis, one can identify related issues and the correct course of action can be taken.

The statistical data analysis of different log tables is helpful in this regard. Both predictive and deterministic analyses are used to precisely locate performance issues. If an issue is not creating a major performance bottleneck, a system study and analysis can identify the problem and solve it.

The client-to-server data exchange takes place when a client connects to the server through a socket. A socket pair, in this case, consists of a client IP address, a client port number, a server IP address, and a server port number. Querying information by the socket therefore lets you zero-in quickly on a specific service running at a specific IP address.

```
$ netstat -antplF
```

The various handshaking communication protocol states are CLOSED, SYN_SENT, ESTABLISHED, FIN_WAIT_1, FIN_WAIT_2 and TIME_WAIT. A connection state ESTABLISHED means that there is a connection, whereas LISTEN means that the socket is waiting for a connection — one is waiting for a connection to be made while the other is already connected. An ESTABLISHED state means that both the hosts have successfully completed the TCP three-way handshake by a successful transition from SYN_SENT to the ESTABLISHED state. A transition from CLOSED to SYN_SENT happens when the client side sends the TCP SYN request to the server. From the above *netstat* command it is possible to get a detailed and periodic status report of client-server communication. Figure 5 gives a bar-plot of Figure 3 to graphically represent the communication status.

```
# R language script
netstat<- read.table("netstat_2_csv.csv",header=TRUE,sep=",")
head(netstat)
detach(netstat)
attach(netstat)
barplot(
```

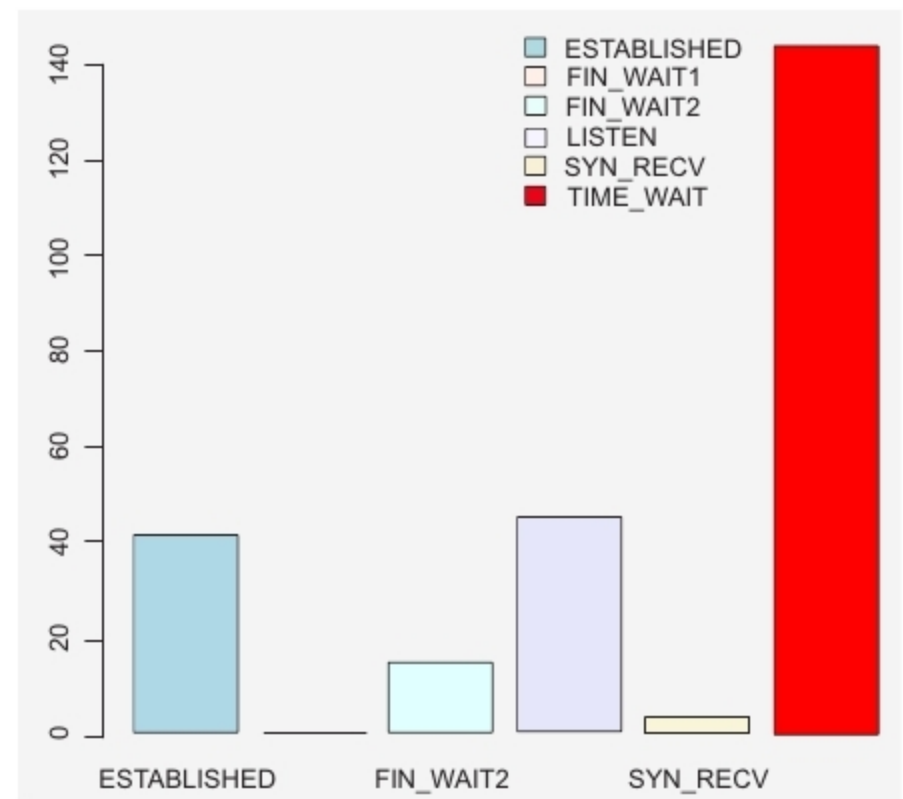


Figure 5: Bar-plot of *netstat* outputs to view the status of different TCP connections

```
table(d),
beside = TRUE,
col = c("lightblue", "mistyrose",
"lightcyan", "lavender", "cornsilk", "red"),
legend = rownames(table(d)), ylim = c(0, 150),
args.legend = list(x = "topright", bty = "n",
inset=c(0.3, 0))
)
```

A summary report of the above log is helpful to gain an in-depth understanding of the system. Figure 4 gives an R summary report of the above *netstat* status log.

If an application is behind a firewall and is in the idle state, then there is a possibility that the firewall may drop the idle TCP connection after a given amount of time. Since there is no way to know the invalid connection, the system will keep the connection forever, and when the client tries to use that connection again, it will be unresponsive.

Acknowledgment: I am thankful to Indrajit Prasad of Wizertech Informatics Pvt Ltd, Kolkata, for his cooperation during the preparation of this article. 

Reference

- [1] <https://serverfault.com/questions/329845/how-to-forcibly-close-a-socket-in-time-wait>

By: Dipankar Ray

The author has more than 20 years of experience with open source versions of UNIX operating systems and Sun Solaris. Presently, he works on data analysis and machine learning using neural networks and different statistical tools. You can contact him at dipankarray@ieee.org.